WEB MEDIUM

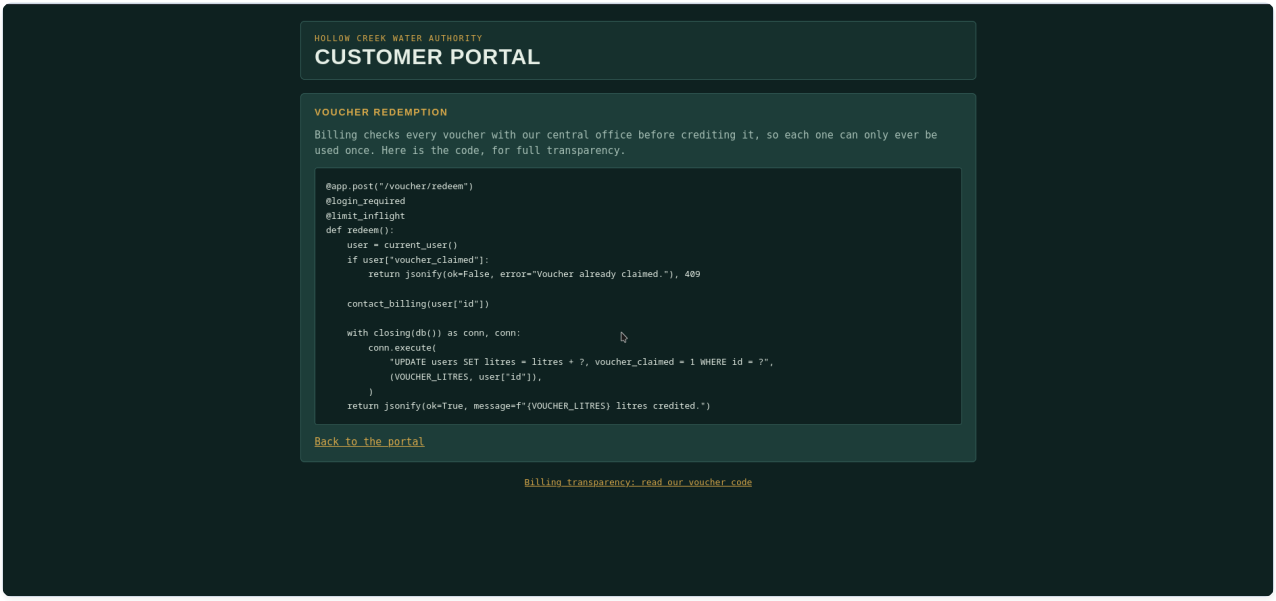
Water Hammer

The web app (`http://web.secleaf.xyz:8880/`) simulates a municipal water meter dashboard. Registering an account gives you a starting balance of 100 litres. There's a button to open the "Reserve Valve" at `/valve/open`, but clicking it complains that you need at least 1,000 litres to unlock it.

There's also a one-time promo voucher worth 50L. Even if you claim it, you're only at 150L, and subsequent claims get rejected with `Voucher already claimed` .

Spotting the Race Window

Browsing to `/billing-code` conveniently showed the backend Python handler for `/voucher/redeem` :



Billing Code Source Leak

Solution Script: Water_Hammer.py (included in attached solution archive)

That `contact_billing()` call sitting squarely between the check (`user["voucher_claimed"]`) and the SQL update is a classic race condition window. The decorator `@limit_inflight` throttles concurrent requests per process, but there is zero row-level DB locking.

If we blast a burst of synchronized requests simultaneously, a bunch of worker threads can evaluate `voucher_claimed == 0` before the first one finishes `contact_billing()` and commits the update.

Exploitation

To ensure all requests hit the server at the exact same millisecond, I used Python's `threading.Barrier` to synchronize 96 concurrent HTTP sessions: **Solution Script:** Water_Hammer.py (included in attached solution archive)

Result

Ran the exploit script:

```
$ python3 solve.py
```



Race Condition Exploitation Proof

Solution Script: `Water_Hammer.py` (included in attached solution archive)

32 requests squeezed into the window before the database locked, catapulting the balance to 1,700L and dropping the flag.

RECOVERED FLAG `SecLeaf{ch3ck_f1r5t_wr1t3_l4t3r_b00m}`